VEOLYN

INTERPRETABILITY QWEN2.5-0.5B-INSTRUCT · HAND-WRITTEN WEIGHTS

A hand-written answer key inside a real model

Most edits reach into the weights a model already has. We did the opposite: 16 hand-written layers in front of Qwen2.5-0.5B, so its own 24 layers never change. The model gained exact multiplication and kept the behaviour it had: on 16 chat prompts the replies are identical, and the logits move by at most 2.8×10^{-4}. Then we pointed the standard tools at a circuit whose answer we already knew: attribution patching missed 13 of the 14 blocks that matter. If our own circuit can slip past them, what about the circuits nobody wrote down?

VEOLYN RESEARCH · NOV 7, 2026 · 29 MIN READ

CONTENTS

00 Introduction

01 Writing a calculator into a chat model

02 Pointing the microscope at it

When the gradient goes flat

Does integrated gradients fix it?

What patching cannot see

03 A switch we wrote, vectors we didn't

Reading the vectors as words

One layer, one position

04 Discussion

Open questions

05 Closing thoughts

00

Introduction

Ask Qwen2.5-0.5B-Instruct what 7311 × 7890 is, and you get a tidy answer in exactly the right format. Only the number is wrong (Figure 1). That is typical for a model this small. On products whose larger number has three to six digits, it gets 2 out of 120 right.

user › What is 7311 × 7890?

QWEN2.5-0.5B-INSTRUCT ✗ 56,245,990

To calculate the product of \(7311 \times 7890\), we can use the standard multiplication method or a calculator for accuracy.

Using a calculator:
\[7311 \times 7890 = 56245990 \]

So, the result of \(7311 \times 7890\) is **56245990**.

SAME MODEL + 16 HAND-WRITTEN LAYERS ✓ 57,683,790

To calculate the product of \(7311 \times 7890\), we can use the standard multiplication method or a calculator for accuracy.

Using a calculator:
\[7311 \times 7890 = 57683790 \]

So, the result of \(7311 \times 7890\) is **57683790**.

FIGURE 1

The same question, before and after. Both replies are real greedy outputs. The text is identical down to the last character except for the product, because the hand-written layers only touch the digits after "=". Everything else is the original model talking.

We fixed this without any training. We put 16 new layers in front of the model and set their weights by hand, so that they compute the product exactly and feed the digits to the model as it writes them. The original 24 layers are untouched, and the fixed model gets 90 of the 120 products right. In the other 30 it works the product out longhand, never writing the "A × B =" pattern the calculator waits for. Each of the 124 times a reply did write that pattern, the product that followed was exact.

If all you want is a model that multiplies, give it a Python tool; that is simpler and more general. We wanted something else. Building the calculator was the fun part, but this post is about a side effect of it. We now have a real, pretrained chat model in which one circuit is completely understood. For every weight in it, we can say which formula it came from and what it is for. That makes the model a good place to test interpretability tools.

Interpretability tools make claims like “this attention layer moves the number to the last token” or “these five blocks are the circuit for this task.” In a trained model, such claims are hard to check, because nobody knows the right answer. Usually the only evidence that a circuit exists comes from the same kind of tools we would like to check. One way around this is to build models whose circuits are known ahead of time. Tracr compiles small programs straight into transformer weights [9: Lindner et al, 2023], and InterpBench trains small transformers to follow known algorithms [4: Gupta et al, 2024]. The Tracr authors say that doing interpretability without such models is like “trying to invent a microscope lens without ever being able to point it at familiar, well understood shapes.”

Those models are small and built from scratch, so there is nothing else going on inside them. Our calculator lives inside a real model, with its real tokenizer, its real normalization, and 24 layers of learned features around it. If a tool can’t find a circuit that we wrote down, in conditions this close to the real thing, we should be careful about what it says about circuits nobody wrote down.

We also gave the model a second, looser kind of mechanism: slash commands like `/zh`, “reply in Chinese.” The switch is hand-written, but the vectors it turns on were measured from the model itself. So for the commands we know exactly when and where something enters the model, but not what it means, which is closer to where interpretability usually works.

WE FIND THAT

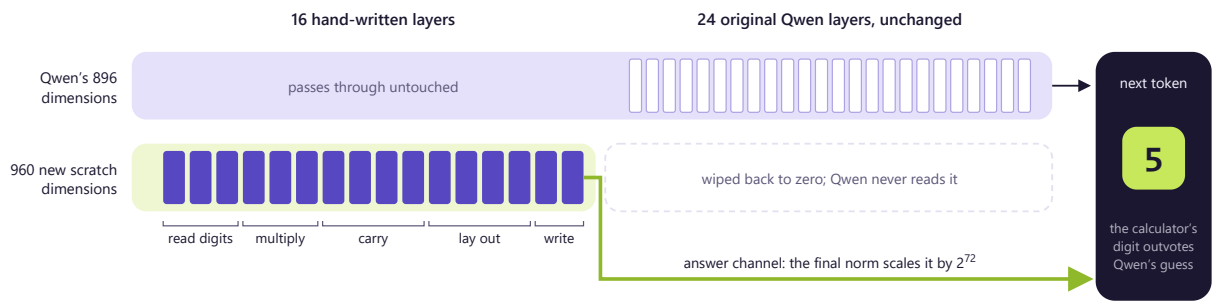
- **Activation patching, the standard way to ask which parts of a model matter, finds the calculator layer by layer.** It also correctly reports that the 24 original layers play no part in the answer.
- **Attribution patching, a popular fast approximation of it, misses 13 of the 14 hand-written blocks that matter.** The calculator makes its decisions with step functions, and a step function is flat almost everywhere, so a method built on gradients sees nothing.
- **Integrated gradients rescues the shortcut on the original Qwen, but not on the calculator.** On any single prompt, its estimate for a hand-written block is usually exactly zero and occasionally ten times too large. Averaged over prompts, it can look right by accident.
- **Patching only sees what the two prompts being compared disagree about.** Two of our layers are essential. Zero out either one and every product comes out wrong, yet patching gives both a score of zero.
- **Reading a vector as a list of words works for style and fails for language.** The `/pirate` vector says "Arr." The `/zh` vector says nothing about Chinese, even though `/zh` is the most reliable command.
- **Nearly all of what `/zh` does happens in one layer, at one position:** the moment the model picks the first word of its reply.

01

Writing a calculator into a chat model

A transformer is built for soft, approximate computation, and three of its parts get in the way of exact arithmetic. Before every layer, a normalization step (RMSNorm) divides each token's vector by its overall size, so a number stored in that vector doesn't stay the same number. The MLPs use SiLU, a smooth curve, where arithmetic needs exact products and sharp yes-or-no decisions. And attention finds tokens by similarity and by a rotary position code [18: Su et al, 2021], while a calculator needs "the token exactly four places back."

Each obstacle has a workaround that uses only the model's own parts. Figure 2 shows how they fit together.



scroll →

FIGURE 2

The grafted model, end to end. Each token carries a vector from layer to layer. We widen it with 960 new dimensions that the original layers never read. The 16 new layers do all their work there. They write the answer digit into a separate channel and wipe everything else back to zero before Qwen's layers run. The digit shown is the first digit of 7311 × 7890.

The vector each token carries between layers is called the residual stream. We make it wider, going from 896 to 1,856 numbers. The original layers have no weights that read the 960 new numbers, so the calculator can use them as scratch paper. One of them holds a huge constant, 2^{24}, in every token. It dominates the vector's size, so the normalization divides every token by almost exactly the same amount, and the next layer can undo that division.

For arithmetic, each neuron in Qwen's MLP computes `silu(gate) * up`. Two of these neurons with opposite signs add up to an exact product. Two more make a step, a function that is 0 for negative inputs and 1 for positive ones:

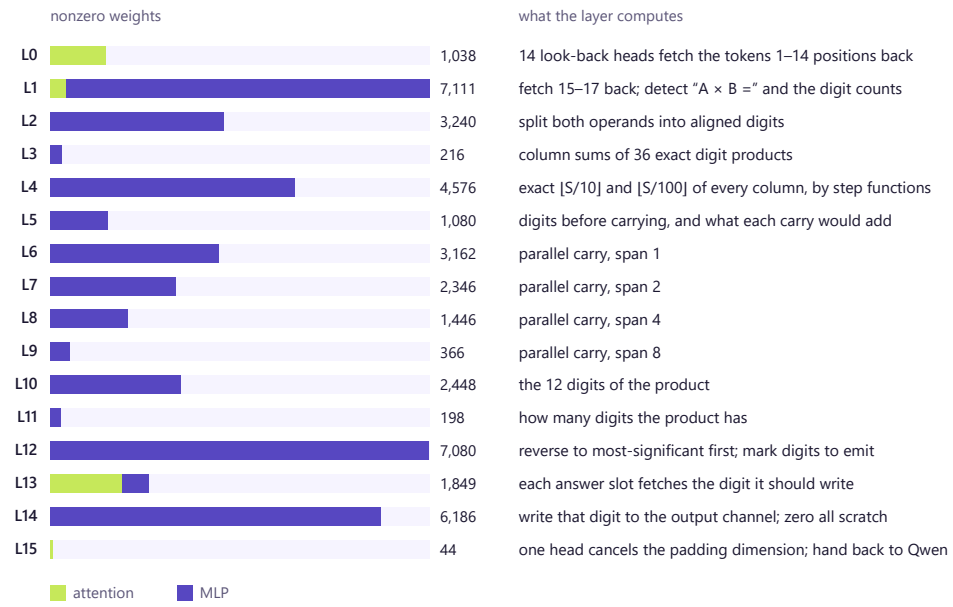
```
# each MLP neuron computes silu(gate) * up

def product(g, u):
# exactly g * u, because sigmoid(g) + sigmoid(-g) = 1
return silu(g) * u + silu(-g) * (-u)

def step(z):
# 1 if z > 0, else 0; the circuit only feeds it z = ±0.5, ±1.5, ...
return (silu(256 * z + 64) - silu(256 * z)) / 64
```

For lookups, we solved a small linear program to choose each attention head's rotary settings. The resulting head attends to the token exactly *k* places back. Its score there beats the score at every other offset by at least 25.9%, out to 40,000 positions. Once scaled, the

attention on every other token is so small that it rounds to exactly zero in float32, so each head copies exactly one token. Carrying uses a Kogge–Stone parallel prefix [7: Kogge & Stone, 1973], a trick from chip design that settles 12 carries in 4 layers instead of 12. The finished digit goes into the answer channel, and the final normalization multiplies that channel by 2^{72}, enough to outvote whatever the original model wanted to say. All of this relies on float32 arithmetic with TF32 turned off. Figure 3 lists what each layer does.



scroll →

FIGURE 3

The 42,386 weights we wrote, layer by layer. The 16 new layers have 494 million parameters, and 0.009% of them are nonzero. Every new dimension has a name in `circuit_layout.json`, so each weight reads as a sentence: this neuron adds this quantity to that one.

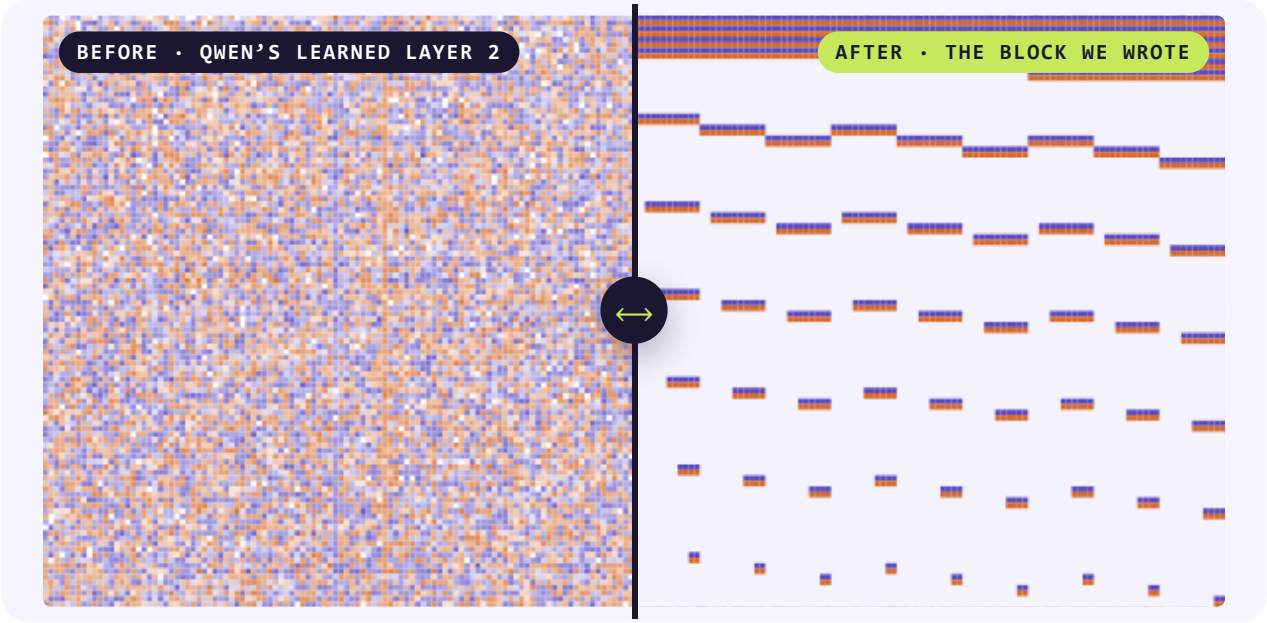
You can look at the weights themselves in Figure 4. Pick a layer, then drag the divider: on one side is a block of the weights Qwen learned for that layer, on the other the block we wrote for it, cut to the same size. Hover over a cell, or tap it on a phone, to see what it connects. On the learned side, nothing has a name.

L2 Line up the digits | L3 36 exact products | L10 Clean up the digits | L13 Fetch digit t

Layer 2: Line up the digits

Each neuron fires for one way the two numbers can sit in the text, and copies one digit into its column.

BEFORE BOTH AFTER



Learned: 23,328 of 23,328 nonzero, 2,120 distinct values
Written: 3,024 nonzero (13.0%), 2 distinct values

FIGURE 4

Learned weights and written weights, side by side. Each hand-written block is every nonzero row and column of one matrix in the graft. The learned block is the same matrix type from the same layer of the original Qwen, the same number of rows and columns from the top left. Purple is positive and orange negative, on a log scale; on the written side, pale cells are exactly 0. Rows are MLP neurons or query dimensions, and columns are dimensions of the residual stream; tall blocks are drawn on their side.

Outside of multiplication, the model behaves exactly as it did before. On 16 chat prompts without a product in them, the greedy replies are identical and the logits differ by at most 2.8×10^{-4}. There is one exception, which we designed in: a reply containing the equation $2 \times 1 = 2$. There the calculator takes over and writes the same digit the model would have written.

For this post, what matters is that every part of the calculator has a known job. So we can check what the tools say about it.

02

Pointing the microscope at it

The most trusted tool for finding the parts of a model that matter is activation patching [23: Zhang & Nanda, 2024]. It is easiest to explain with our own prompts. We take two prompts that differ in one digit, and cut each off right where the model is about to write the first digit of the answer:

```
CLEAN      Using a calculator: \[ 599748 \times 56930 =   3
CORRUPTED  Using a calculator: \[ 599748 \times 46930 =   2
```

We run both. Then we run the corrupted prompt again, but at one block we paste in the output that block produced on the clean prompt (Figure 5). If the model now writes 3 instead of 2, that block carries the information that tells the two prompts apart. Doing this for every attention block and every MLP block gives a map of where that information flows.

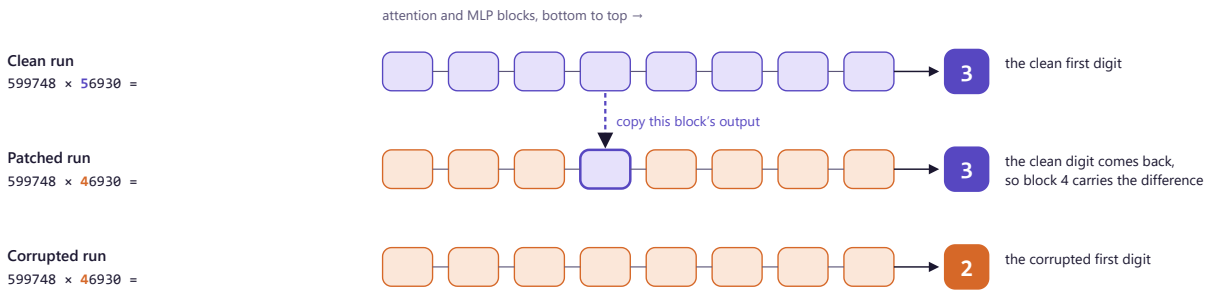


FIGURE 5

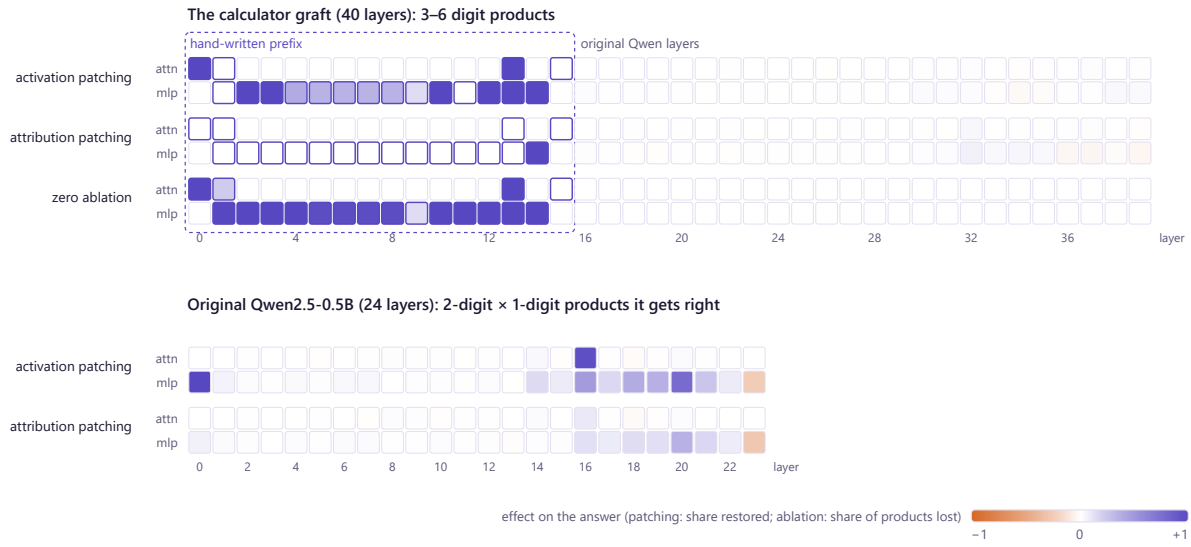
Activation patching, on one of our prompt pairs. The patched run sees the corrupted prompt everywhere except at one block, whose output is copied from the clean run. Repeating this for each block tells you which ones the difference passes through.

The prompt text isn't made up. Before it gives a product, the model writes this line, as in Figure 1:

```
Using a calculator: \[ A \times B =
```

LATEX, AS THE MODEL WRITES IT

We follow the usual recommendations [6: Heimersheim & Nanda, 2024]. The digit we change is chosen so that the first digit of the answer changes too. We measure how strongly the model prefers one digit over the other. A score of 0 means patching left the corrupted preference as it was, and 1 means it brought the clean preference fully back. We average over 24 pairs, with three to six digits per number, and patch all 80 blocks in the 40 layers. Figure 6 shows the result, next to two other ways of asking the same question.



scroll →

FIGURE 6

Which blocks matter, asked three ways, on a model where we know the answer. Each cell is one attention or MLP block, averaged over 24 prompt pairs; purple outlines mark blocks that contain hand-written weights. Top: the grafted model. Activation patching lights up the hand-written chain and nothing else. Attribution patching sees only the last MLP. Zero ablation, the share of products that come out wrong when a block's output is set to zero, also catches blocks that patching misses. Bottom: the original model, on products it can do.

The top row of Figure 6 is what we hoped to see. Patching any of the following fully restores the clean answer:

- the first attention layer, which fetches the digits;
- the MLPs that split the numbers into digits and add up the columns (layers 2 and 3);
- the MLPs that tidy the digits and reverse their order (10 and 12);
- the last steps in layers 13 and 14, which fetch each answer digit and write it out.

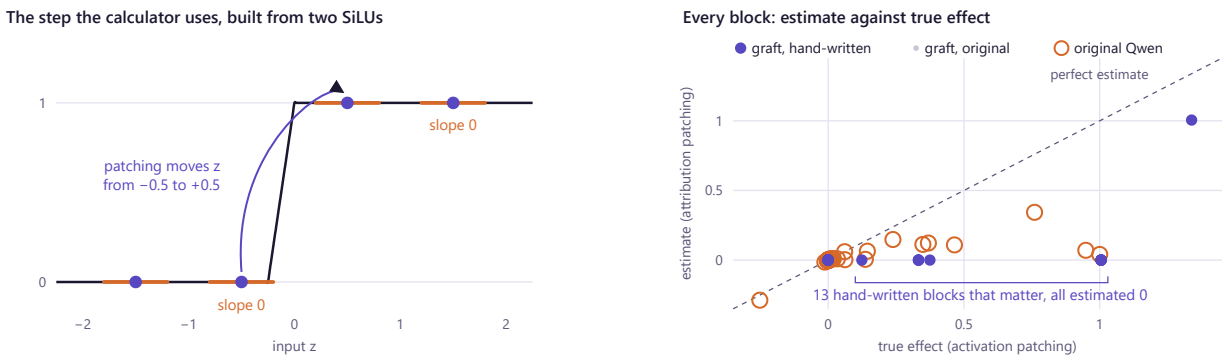
The 24 original layers make up 60% of the network's depth, and they score zero. That is correct: in the grafted model they have nothing to do with the answer.

The carry layers, 4 through 9, score between 0.12 and 0.37, and that average hides something odd. Take layer 5. In 4 of the 24 pairs, patching it brings back the clean answer. In 12 the corrupted answer stays. In the other 8 the model writes a third digit that is neither. The third digit comes from a mix the model would never see on its own: correct carries from one layer combined with wrong numbers everywhere else. Patching tries to measure one block's contribution. When the steps run one after another, though, one step's contribution only makes sense relative to the steps around it.

When the gradient goes flat

Activation patching needs one extra run of the model per block. For big models and many prompts that adds up, so people often use a shortcut called attribution patching [11: Nanda, 2022]. One backward pass gives the gradient of the answer with respect to every block at once. Multiply each block's gradient by how much its output differs between the two prompts, and you have an estimate of what patching that block would do. It is a straight-line guess: it assumes the answer changes in proportion to the change in the block.

On the calculator, the guess is zero for every block except the last (Figure 7, right). The step function explains why (Figure 7, left): the calculator only ever feeds it values like −0.5 or +0.5. Those sit away from the jump at zero, where the curve is perfectly flat. On a flat stretch the gradient is zero, which correctly says that nudging the input changes nothing. But patching doesn't nudge. It moves the input from −0.5 to +0.5, straight across the jump, and the output goes from 0 to 1. Every hand-written block except the last reaches the answer through at least one step, so each of them gets an estimate of zero. The last MLP is the exception because it writes the digit straight into the output. Its estimate is 1.00 and its true effect is 1.34. A score above 1 means the patched run prefers the clean digit even more strongly than the clean run does, which can happen when the rest of the network is still working from corrupted values.



scroll →

FIGURE 7

Why the gradient sees nothing. Left: the step function from the code above. The calculator only evaluates it on flat stretches (dots), where the slope is zero, but patching moves the input across the jump. Right: for every block in both models, the attribution estimate against the true effect from activation patching. Points on the dashed line would be perfect estimates. The hand-written blocks sit on the floor; the two orange rings near the purple dots at (1, 0) are original-Qwen blocks the shortcut also misses.

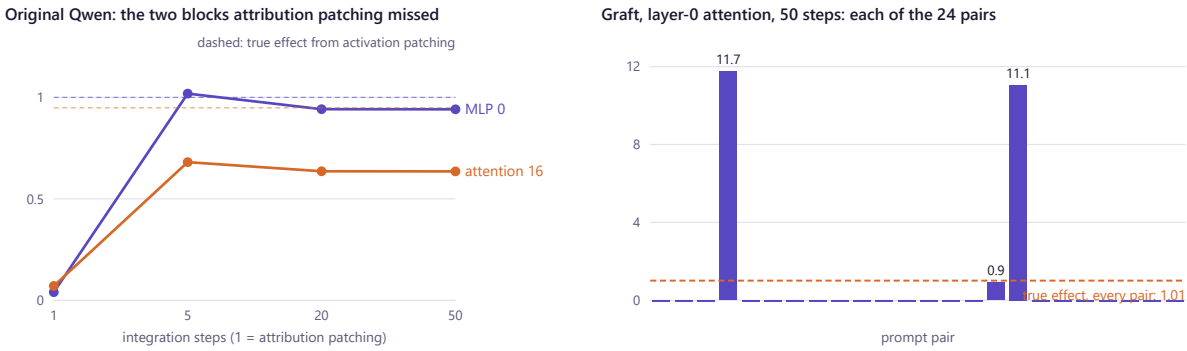
You could fairly say we built a worst case. So we ran the same comparison on the original Qwen, using 2-digit by 1-digit products it gets right (orange rings in Figure 7, bottom of Figure 6). Here the shortcut does better, but not well. It sees the last few MLPs while underestimating them: 0.34 for layer 20 against a true effect of 0.76, and 0.11 for layer 16 against 0.46. And it misses the two blocks that matter most. The MLP in layer 0 has a true effect of 1.00 and an estimate of 0.04. The attention block in layer 16, which moves the changed digit to the last token, has a true effect of 0.95 and an estimate of 0.07. AtP* describes this failure in trained models. When a nonlinearity sits on a flat stretch, as in an attention softmax that has saturated, the local slope says little about what a large change would do [8: Kramár et al, 2024]. Our calculator makes the failure total.

Put together, the two models suggest where gradients can be trusted: close to the output, where the computation is nearly linear. Choosing which digit to write, which token to copy, or whether to carry a one is the kind of job that flat-then-steep nonlinearities do. That is where the shortcut breaks.

Does integrated gradients fix it?

The standard answer to a flat gradient is to stop asking for the slope at one point. Integrated gradients walks along a path from the corrupted input to the clean one, takes the gradient at many points along the way, and averages them [19: Sundararajan, Taly & Yan, 2017]. Circuit

discovery uses the same idea under the name EAP-IG [5: Hanna, Pezzelle & Belinkov, 2024]. We ran it on both models, interpolating the input embeddings and averaging over 1, 5, 20 or 50 points (Figure 8). With one point, it is exactly attribution patching.



scroll →

FIGURE 8

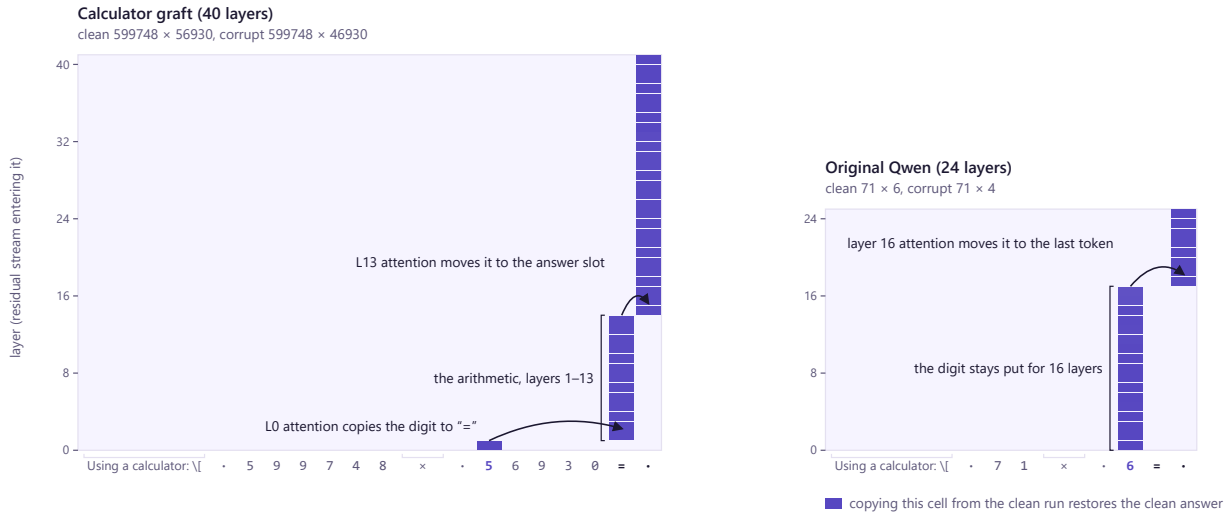
Integrated gradients rescues the trained model, not the calculator. Left: on the original Qwen, five points along the path are enough to recover most of the two blocks that attribution patching missed. Right: on the graft, the estimate for the first attention layer on each of the 24 pairs with 50 points. It is exactly zero on 21 pairs and far too large on most of the rest, though the true effect is 1.01 on every pair.

On the original Qwen it works. With five points, the estimate for the layer-0 MLP goes from 0.04 to 1.02, and the one for the layer-16 attention goes from 0.07 to 0.68; more points change little. A trained model's flat stretches are flat only locally, and somewhere along the path the gradient wakes up.

On the calculator it doesn't, and the way it fails is instructive. For the first attention layer, the average over the 24 pairs is 0.99 with 50 points, which looks right. Pair by pair, it is exactly zero on 21 of them and 11.75, 11.06, 0.91 on the others. With 20 points the average is 0.07. The jump of a step is narrow, and all of its effect lives there. A path of 50 points usually steps over it. When a point does land on it, that one sample is counted as if the slope held for a fiftieth of the path, which overshoots by an order of magnitude. The method is right on average and wrong on almost every single prompt. Further from the input it gets worse. The carry layers and the blocks that lay out and write the answer sit behind several steps, which would all have to be mid-jump at the same point on the path; their estimates stay at 0.06 or below.

What patching cannot see

Patching can also follow the information token by token. Instead of patching whole blocks, we patch the residual stream at one token as it enters one layer (Figure 9). In the grafted model, the route is the one we designed. The first attention layer copies the changed digit onto the `=` sign. It sits there through all of the arithmetic, and the attention in layer 13 moves it to the position where the answer is written. The original model does it differently. It leaves the digit where it is for 16 layers, then moves it straight to the last token.



scroll →

FIGURE 9

Where the difference between the two prompts lives. Columns are the prompt's tokens (· is a space) and rows are layers, from the bottom up. A cell is colored if copying the residual stream at that token, entering that layer, from the clean run into the corrupted run restores the clean answer. The grafted model moves it twice, at the layers we wrote for it; the original moves it once, two-thirds of the way up. Each panel shows one pair; all 24 pairs follow the same route.

Some layers we wrote never appear in this picture. Layer 1 checks whether the text is a multiplication at all and how many digits each number has. Layer 11 works out how many digits the answer has. Both are essential: zero out either one's MLP and every product comes out wrong, as the zero-ablation row of Figure 6 shows. Yet both score zero under patching. Our clean and corrupted prompts have the same format and the same answer length, so these layers compute the same thing on both, and there is no difference for patching to restore.

So patching finds the parts that carry the difference between two inputs. That set is smaller than the parts the computation needs, and which subset you see depends on the difference you chose. Heimersheim and Nanda make this point in general terms. An answer key makes it concrete. If you build a circuit diagram out of patching experiments, you only get the parts that the contrasts you happened to try can reach.

03

A switch we wrote, vectors we didn’t

The calculator is the easy case, since everything about it is written down. Our second mechanism is closer to what interpretability usually deals with.

We gave the model four slash commands. `/zh` makes it reply in Chinese, and `/brief`, `/cheer` and `/pirate` do what their names say. A command can go anywhere in a message and stays on for the rest of the conversation. The switch is hand-written: the new front layers recognize each command’s exact tokens and set a flag. Each original layer gets a few extra attention heads that look only at those flags. Without a command, the flags are zero. The heads then add exactly zero, and the output is bit-for-bit what it would be without them. With a command, one head adds a vector at every later token. Another adds a different vector at the token where the model picks the first word of its reply. Figure 10 shows this on a real conversation.



FIGURE 10

What `/zh` adds, token by token. One real conversation with the command, colored by what the mode heads add at each token, summed over the 24 original layers. Hover over a token for the numbers. Before the command, the heads add exactly zero. After it, the every-token vector goes in everywhere, and the first-token vector goes in at one position only, just before the model writes its first word. Right: the model's top choices for that first word, without and with the command.

The vectors themselves are not hand-written. We measured them on the original model. For `/zh`, we asked a set of questions with "请用简体中文回答" ("please answer in Simplified Chinese") added to each one. At each layer, we recorded how much the attention output changed on average over the model's own Chinese replies. For `/cheer`, the instruction was a system message asking for emojis and exclamation marks. Attention is the only way an instruction earlier in the context can affect later tokens. So these vectors are, roughly, the instruction compiled into a fixed addition. They are close relatives of steering vectors `[15,21:`

Rimsky et al, 2024; Turner et al, 2023], function vectors [20: Todd et al, 2024], and context distillation [17: Snell, Klein & Zhong, 2022]. The difference is that nothing is trained, and the vectors sit behind an exact switch.

So for these commands we know exactly when and where each vector goes in, and nothing about what it means. How much can the usual tools tell us?

Reading the vectors as words

The logit lens takes a vector from inside the model and passes it through the model's last normalization and output layer. That tells you which words the vector pushes up [13: nostalgebraist, 2020]. Figure 11 does this for each command's two vectors, summed over the 24 layers.

/pirate (talk like a pirate)	first	`·Arr` `Arr` `·arr` `arr` `_arr` `(arr` `⇥arr` `ARR` `.arr` `[arr`
	reply	`ye` `·ye` `!` `-` `·y` `·yer` `yy` `'` `·Arr` `Arr`
/cheer (be upbeat)	first	`·emoji` `满满` `ost` `不停` `·punct` `·hashtag` `·�` `·hashtags` `·�` `·confirmed`
	reply	`·�` `·�` `!` `·@` `!!` `)!` `!↲` `·�` `!!!` `!↲↲`
/brief (be short)	first	`·const` `·never` `never` `·Never` `Never` `:` `·C` `<|endoftext|>` `...` `,`
	reply	`<|endoftext|>` `.'` `.</` `.`` `>.</` `'.` `..` `...` `.)` `·post`
/zh (reply in Chinese)	first	`·beating` `周` `·beat` `넜` `[][]` `========↲` `一遍` `};↲↲↲` `:event` `ponsor`
	reply	`"".` `⬇` `полнить` `kg` `).↲↲↲↲` `·Archived` `☞` `⨯` `.masksToBounds` `·Aç`
/zh, layer 17 only	first	`oldem` `nyder` `FINE` `ucson` `#af` `(KP` `owntown` `/lg` `incinn` `mdat`

FIGURE 11

The words each command's vectors push up most, under the logit lens. "first" is the vector added where the first word of the reply is chosen; "reply" is added at every token after the command. · marks a leading space, ↲ a newline, and `�` a byte fragment of a multi-byte character such as an emoji. The last row is the single most important `/zh` weight (Figure 12).

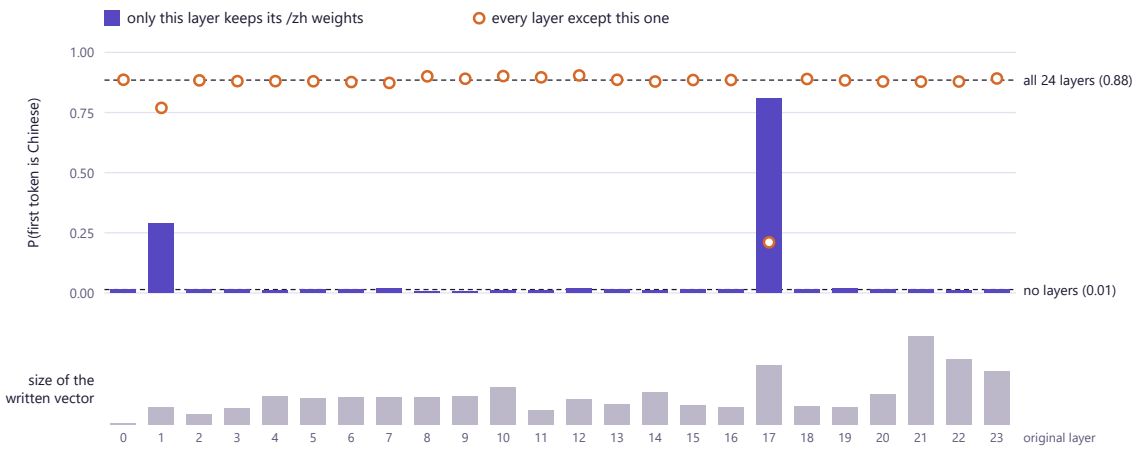
The style commands read like their names. `/pirate`'s first-word vector pushes up "Arr" in a dozen spellings, and its every-token vector pushes up "ye" and "yer." `/cheer` pushes up exclamation marks, pieces of emoji, and the words "emoji" and "hashtag." `/brief` pushes up end-of-text and closing punctuation, which is a fair summary of being brief. These vectors work the way the lens assumes: they vote directly for words.

`/zh` does not. Its vectors push up a scatter of symbols, English fragments, Cyrillic, Korean, and a few unrelated Chinese characters. Its single most important weight, in layer 17, pushes up no Chinese token at all. Yet `/zh` is the most reliable of the four commands. Todd et al. saw something similar with function vectors, where the top decoded words are usually not

enough to rebuild a vector that works. A vector that says “Arr” is a vote for a word. A vector that says “answer in Chinese” seems to work through the layers after it, and a readout that skips those layers can’t see what it does.

One layer, one position

Because we know exactly where the `/zh` weights are, we can take them apart. Figure 12 keeps them in a single original layer, or in all layers but one. It then measures how likely the reply is to start with a Chinese token, averaged over 38 English questions the vectors were not measured on.



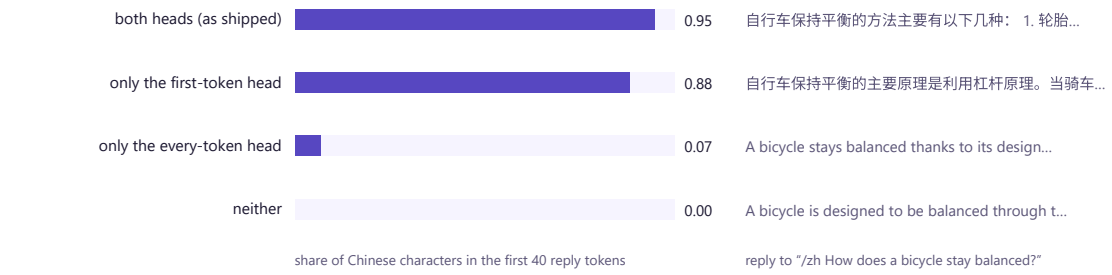
scroll →

FIGURE 12

The `/zh` weights, one layer at a time. Top: the probability that the first reply token is Chinese when only one original layer keeps its `/zh` weights (bars), or when every layer but one does (circles). Bottom: the size of the first-token vector in each layer. Original layer 17 is layer 33 of the grafted model.

Layer 17 does almost all the work. On its own it gives 0.81, against 0.88 with all 24 layers and 0.01 with none. Without it, the other 23 layers reach only 0.21. Layer 1 is a distant second at 0.29, and no other layer reaches 0.03 alone. Each of the 24 vectors is just the change the instruction causes at that layer, and most of those changes turn out not to matter for choosing the language. Layer 17's vector is not even the biggest; the ones in layers 21 and 22 are larger. Judging by the size of the weights would have pointed to the wrong layer.

The two heads can be separated too. Figure 13 keeps only one of them and lets the model write 40 tokens.



scroll →

FIGURE 13

The first-word head does the switching. With only the head that acts where the first reply token is chosen, 0.88 of the reply's characters are Chinese. With only the head that acts at every token, 0.07 are.

The language gets chosen once: the first-word vector makes the first token Chinese, and from then on the reply so far keeps the model in Chinese. Adding the every-token vector only raises the share of Chinese characters from 0.88 to 0.95. We can say this with confidence only because we know the vectors go in at exactly two kinds of position and nowhere else (Figure 10). In a trained model it is hard even to ask whether a behavior is decided at one position or kept up at every position, because we rarely know where it enters in the first place.

04

Discussion

In this testbed, activation patching was never wrong about what it measures. The trouble was in reading it: mixed scores when steps run one after another, and blind spots wherever the two prompts don't differ. Attribution patching was simply wrong about every hand-written block but the last. Integrated gradients fixed it on the trained model and turned it into a lottery on the calculator. Fast approximations are necessary at scale, and AtP* fixes some of their misses in trained models. Neither of its fixes targets an MLP sitting on a flat stretch, though. One recomputes attention softmaxes, and the other drops gradients through single layers to expose cancellations. The lesson we take is narrow: gradient methods inherit the smoothness of the model they run on. Learned networks tend to be smooth enough. A network that makes hard decisions, by design or because training pushed it there, may not be.

The commands show that a mechanism can be fully legible while its content is not. By construction, `/zh` adds exactly zero before the command and exactly two vectors after it. What those vectors represent, we can only describe by what they do. We think this split is a reasonable way to build systems, not just a curiosity. Write by hand the parts that need guarantees, such as when a behavior is allowed to turn on, and measure or learn the parts that need to be rich.

More broadly, answer keys don't have to live in toy models. Grafting a known circuit into a pretrained model takes no training and leaves the rest of the model alone. Any interpretability method can then be graded on it with a real tokenizer, real normalization, and the model's own features all around.

There are clear limits. Our calculator lives in its own 960 dimensions, apart from everything the original model represents. Trained circuits share dimensions with many other features in superposition [2: Elhage et al, 2022], so a tool that finds our circuit might still fail on theirs. The flatness that defeats attribution patching is extreme by design. And the experiments are small: one model, 24 prompt pairs for patching, and 38 questions for the commands.

Open questions

- Can a hand-written circuit share dimensions with the model's own features and still be exact? That would test the case that matters most.
- Attribution graphs built from transcoders explain addition in Claude 3.5 Haiku as two pathways, one estimating the rough size of the sum and one getting the last digit right [10,1: Lindsey et al, 2025; Ameisen et al, 2025]. Would they recover a multiplier we wrote, carries and all?
- Why layer 17? Is it where Qwen commits to an output language, and do other languages go through the same layer?
- How saturated are trained models in practice? If some of their circuits make decisions as sharply as ours, gradient-based circuit discovery would miss them in the same way, and we would not know.

05

Closing thoughts

Trained models are messy, and it is tempting to conclude that interpretability methods can only be tested on toys or taken on trust. We don't think those are the only options. A pretrained model has room for circuits we fully understand, sitting next to the ones we don't. Learned arithmetic in language models looks like a bag of heuristics [12: Nikankin et al, 2025], and recent work trains models sparse enough that their circuits can be read [3: Gao et al, 2025]. Writing the circuit by hand is the far end of that range, where the explanation comes first and the weights follow from it. Before we trust a method on a model where nobody knows the answer, we should grade it on one where somebody does.

Acknowledgements

We thank the Qwen team for releasing Qwen2.5-0.5B-Instruct under Apache-2.0. We also thank the authors of Tracr, ALTA [16: Shaw et al, 2025], and Thinking Like Transformers [22: Weiss, Goldberg & Yahav, 2021], whose compiled transformers inspired this work. Every number in this post comes from scripts in the project repository (`analysis/patching.py`, `analysis/ablate.py`, `traits/lens.py`, `traits/anatomy.py`), which write the JSON files the figures are drawn from. The code is open source, work in progress; this page will link to the repository as soon as it is public.

The experiments in this post were run in Veolyn's research environment, with Opus 5.5 leading the work.

Citation

Please cite this work as:

```
Veolyn Research, "A hand-written answer key inside a real model",
Veolyn Research, Nov 2026.
https://veolyn-ai.github.io/blog/llm-weight-edit-interpretability/
```

Or use the BibTeX citation:

```
@article{veolyn2026answerkey,
author  = {{Veolyn Research}},
title   = {A hand-written answer key inside a real model},
journal = {Veolyn Research},
year    = {2026},
month   = {nov},
url     = {https://veolyn-ai.github.io/blog/llm-weight-edit-interpretability/},
}
```

References

1. Ameisen, E., Lindsey, J., et al. Circuit Tracing: Revealing Computational Graphs in Language Models. Transformer Circuits Thread, 2025.
2. Elhage, N., et al. Toy Models of Superposition. Transformer Circuits Thread, 2022.
3. Gao, L., Rajaram, A., Coxon, J., Govande, S., Baker, B., Mossing, D. Weight-sparse transformers have interpretable circuits. arXiv:2511.13653, 2025.
4. Gupta, R., Arcuschin, I., Kwa, T., Garriga-Alonso, A. InterpBench: Semi-Synthetic Transformers for Evaluating Mechanistic Interpretability Techniques. arXiv:2407.14494, 2024.
5. Hanna, M., Pezzelle, S., Belinkov, Y. Have Faith in Faithfulness: Going Beyond Circuit Overlap When Finding Model Mechanisms. COLM, 2024.
6. Heimersheim, S., Nanda, N. How to use and interpret activation patching. arXiv:2404.15255, 2024.
7. Kogge, P. M., Stone, H. S. A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations. IEEE Transactions on Computers, 1973.
8. Kramár, J., Lieberum, T., Shah, R., Nanda, N. AtP*: An efficient and scalable method for localizing LLM behaviour to components. arXiv:2403.00745, 2024.
9. Lindner, D., Kramár, J., Farquhar, S., Rahtz, M., McGrath, T., Mikulik, V. Tracr: Compiled Transformers as a Laboratory for Interpretability. NeurIPS, 2023.
10. Lindsey, J., et al. On the Biology of a Large Language Model. Transformer Circuits Thread, 2025.
11. Nanda, N. Attribution Patching: Activation Patching At Industrial Scale. 2022.
12. Nikankin, Y., Reusch, A., Mueller, A., Belinkov, Y. Arithmetic Without Algorithms: Language Models Solve Math With a Bag of Heuristics. ICLR, 2025.
13. nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020.
14. Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
15. Rimsky, N., Gabrieli, N., Schulz, J., Tong, M., Hubinger, E., Turner, A. Steering Llama 2 via Contrastive Activation Addition. ACL, 2024.
16. Shaw, P., Cohan, J., Eisenstein, J., Lee, K., Berant, J., Toutanova, K. ALTA: Compiler-Based Analysis of Transformers. TMLR, 2025.
17. Snell, C., Klein, D., Zhong, R. Learning by Distilling Context. arXiv:2209.15189, 2022.
18. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.
19. Sundararajan, M., Taly, A., Yan, Q. Axiomatic Attribution for Deep Networks. ICML, 2017.
20. Todd, E., Li, M. L., Sharma, A. S., Mueller, A., Wallace, B. C., Bau, D. Function Vectors in Large Language Models. ICLR, 2024.
21. Turner, A. M., Thiergart, L., Leech, G., Udell, D., Vazquez, J. J., Mini, U., MacDiarmid, M. Steering Language Models With Activation Engineering. arXiv:2308.10248, 2023.
22. Weiss, G., Goldberg, Y., Yahav, E. Thinking Like Transformers. ICML, 2021.
23. Zhang, F., Nanda, N. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. ICLR, 2024.